

Koşullar ve döngüler

Koşul ifadeleri, belirli bir şartın doğru yada yanlış olmasına göre programın farklı yönere gitmesini sağlar. C# dilinde temel olarak kullanılan koşul yapıları *if, else, else if* ve *switch case, for, foreach* ve *while* ifadeleridir.

if-else yapısı nedir?

if-else yapısı en temel koşul kontrol mekanizmasıdır. Belirtilen koşul True ise if bloğu çalışır eğer değilse else bloğu devreye girer.

Örnek 1: Klavyeden girilen sayının tek mi, çift mi olduğunu gösteren C# programını yazalım.

```
Console.Write("Bir sayı giriniz:");
int sayi = Convert.ToInt32(Console.ReadLine());
if (sayi % 2 == 0)
    Console.WriteLine("Girdiğiniz sayı çifttir.");
else
    Console.WriteLine("Girdiğiniz sayı tektir.");
Console.ReadKey();
```

else if yapısı nedir?

Bir programda ilk koşul doğru değilse, alternatif bir koşulu kontrol etmek için kullanılan karar yapısıdır.

```
61
62     Console.WriteLine("Lütfen yaşınızı giriniz:");
63     string yasOkunan = Console.ReadLine();
64     int yas = Convert.ToInt32(yasOkunan);
65
66     string message = "";
67
68     if (yas < 18)
69     {
70         message = "Ehliyet alabilmeniz için en az 18 yaşında olmalısınız.";
71     }
72     else if (yas >= 18 && yas <= 60)
73     {
74         message = "Ehliyet sınavına girebilirsiniz.";
75     }
76     else
77     {
78         message = "60 yaş üstü bireyler için ehliyet alınamaz.";
79     }
80
81     Console.WriteLine(message);
```

```
int main() {
```

```
    int time = 16;
```

```
    if (time < 12) {
```

```
        printf("Good morning.");
```

```
    } else if (time < 18) {
```

```
        printf("Good day.");
```

```
    } else {
```

```
        printf("Good evening.");
```

```
    }
```

```
    return 0;
```

switch-case yapısı nedir?

Birden fazla koşulu kontrol ederken switch-case yapısı daha okunabilir ve performanslı [olur.Bu](#) yapıda bir değişkenin değerine göre belirli işlemler yapılır.

Örnek 2: Bugün haftanın hangi günüyse o ekrana yazılsın. İkinci örnekte de girilen gün ekrana yazılsın.

```
Console.WriteLine("Haftanın bir gününü yazınız: ");

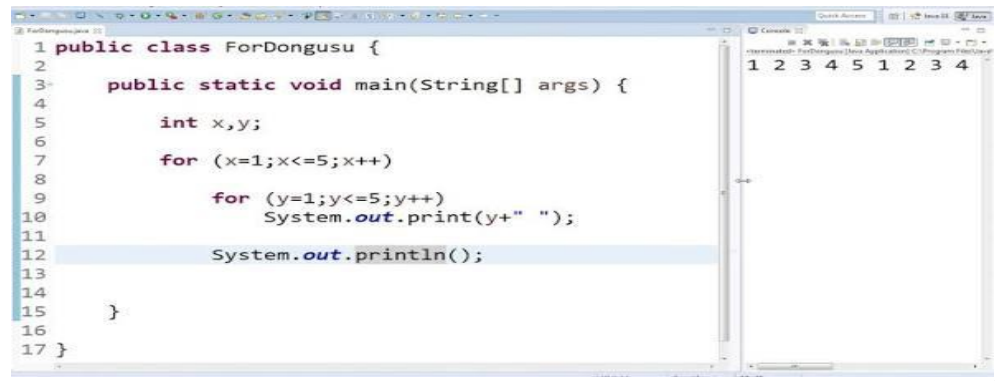
string gun = Console.ReadLine();

switch (gun)
{
    case "Pazartesi":
        Console.WriteLine("Haftanın 1. günü");
        break;
    case "Salı":
        Console.WriteLine("Haftanın 2. günü");
        break;
    case "Çarşamba":
        Console.WriteLine("Haftanın 3. günü");
        break;
    case "Perşembe":
        Console.WriteLine("Haftanın 4. günü");
        break;
    case "Cuma":
        Console.WriteLine("Haftanın 5. günü");
        break;
    case "Cumartesi":
        Console.WriteLine("Haftanın 6. günü");
        break;
    case "Pazar":
        Console.WriteLine("Haftanın 7. günü");
        break;
    default:
        Console.WriteLine("Hatalı gün girdiniz");
        break;
}
```

For döngüsü nedir?

Bilinen sayıda tekrar edilecek işlemler için kullanılan temel bir C# yapısıdır. Bir kod bloğunun çok kez tekrarlamak için kullanılır.

```
for i := 1; i <= 5; i++ {
    fmt.Println(i)
}
```



Foreach döngüsü nedir?

Bir dizi veya koleksiyon içerisindeki tüm elemanları baştan sona sırasıyla ve otomatik olarak dolaşmak için kullanılan bir program yapısıdır. Döngünün ne kadar döneceğini dizinin eleman sayısına bağlı olarak program tarafından otomatik hesaplanacaktır otomatik hesaplanacaktır.

```
using System;

namespace Loop
{
    class ForEachLoop
    {
        public static void Main(string[] args)
        {
            char[] myArray = {'H','e','l','l','o'};

            foreach(char ch in myArray)
            {
                Console.WriteLine(ch);
            }
        }
    }
}
```

```
using System;

namespace Loop
{
    class ForLoop
    {
        public static void Main(string[] args)
        {
            char[] myArray = {'H','e','l','l','o'};

            for(int i = 0; i < myArray.Length; i++)
            {
                Console.WriteLine(myArray[i]);
            }
        }
    }
}
```

While döngüsü nedir?

Önceden belirlenmiş bir koşul bozulana kadar devam eder. Koşulun her kontrolünden sonra döngü içerisindeki işlemler bir kez yapılır. While döngüsünden çıkmak için tek yol başlangıç koşulun bozulması değildir. Döngü içeriden kırılarak da terk edilebilir.

Şart ifadesinin doğruluğu her döngünün başında kontrol edilir.

```
int x = 0;
while (x < 10)
{
    cout << x;
    x++;
    if (x == 5)
        break;
}

int x = 0;
while (x < 10)
{
    cout << x;    x++;
}
```

Do-While döngüsü nedir?

Koşulun doğru olup olmadığına bakmaksızın içindeki kod bloğunu en az bir kez çalıştıran, ardından koşulu kontrol eden programlama yapısıdır. [Koşul](#) yanlış olsa bile, kod bloğu koşul test edilmeden önce yürütüldüğü için döngü her zaman en az bir kez çalıştırılacaktır.

```
int sayi = 0, toplam = 0, i = 1;

do
{
    Console.Write(i + ". sayi: ");
    i++;

    sayi = int.Parse(Console.ReadLine());
    toplam += sayi;
} while (sayi > 0);

Console.WriteLine("Toplam: " + toplam);
```

```
int i = 1;
do
{
    Console.WriteLine(i++);
}
while (i <= 5); // 1'den 5'e kadar ekrana yazar
```